MyCodeSchool

Code review — victor-manual

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-23 — sha c56a122

✅ Toate cele 4 🔴 din runda 1 sunt reparate exact cum cerea fix-ul, într-un singur commit (`c56a122`), și verificate prin rulare pe cazurile care le stricau. Compilează curat cu `g++ -std=c++17 -Wall -Wextra` .

Constatare din runda 1	Rezultat rulat
B1 oglinda (`vectori.h:290`)	✅ închis: `9 721 72 6 108 180 1221 121 537 1000` → `127 72 6 801 180 1221 121 735 1000` (exact ca în enunț); `4 525 72 6 120` → `nu exista` ; `3 505 907 12` → `505 709 12` ; `3 100 200 999` → `nu exista`
B2 căutarea binară (`vectori.h:192–196`)	✅ închis: cu ASan, timeout 3 s: x=0 → `nu exista.` , x=1 → 1, x=5 → 5, x=8 → 6, x=32 → 32, x=100 → 32; se termină de fiecare dată
B3 căutarea liniară (`vectori.h:165` , `i >= 0`)	✅ închis: x=1 → 1 (înainte `nu exista.`), x=0 → `nu exista.` ; identic cu varianta binară pe toate cele 6 valori
B4 inserarea (`vectori.h:117` , `v[1000]`)	✅ închis: `-fsanitize=address,undefined` pe 499 de numere pare → 998 de elemente (499 × 2 și 499 × 2019), niciun raport ASan

0 🔴 deschise. Mai jos: ce a rămas din 🟡 și 🟢 (aproape tot neatins) și două lucruri noi mici.

Tiparul rundei: ai reparat exact liniile semnalate și ai lăsat pe dinafară toate 🟡 .

🔴 Critice

Niciuna deschisă.

🟡 Importante

- **M1** (NEATINS din runda 1) `vectori.h:140`, `solutie7`: tot `d = 7` la 8 valori scrise. Rulat: iese `12 17 32 16 63`, fără `87`; cu `d = 8` (copie în scratchpad) iese corect `12 17 32 16 63 87`.
- **M2** (NEATINS) Niciun `fstream` în `vectori.h` / `manual.cpp` (grep gol). Problemele 1–9 au tot date puse direct în cod, nu din `numere.in` / `bac.txt`, iar 3, 4 și 5 nu scriu în `numere.out`.
- **M3** (NEATINS) `vectori.h:59,78,95`: `v[100]` la problemele 3, 4 și 5, deși enunțul permite 1 000 000 de numere. Vectorul de frecvență ar trebui umplut direct la citire. 💡 Citește fiecare număr într-o variabilă și pune-l direct în vectorul de frecvență.
- **M4** (repetat) `manual.cpp:7`: `main()` cheamă tot o singură funcție (acum `solutie6`, în loc de `solutie9`). Repararea s-a făcut fără să fie rulate `solutie8`, `solutie8alternativ` și `solutieBAC`; le-am rulat eu și ies corect. 💡 Un apel la fiecare `solutieX()` și rulat tot înainte de commit.
- **M5** (NEATINS) `vectori.h:34`: problema 2 cu `n = 0` afișează `a)` și `b)` goale, nu `0` (rulat: `a) \nb) <END`).

🟢 Cleanups

- **C1** (NEATINS) `vectori.h:162,182`: `int x = d;`. `x` e valoarea căutată și coincide doar întâmplător cu `d`.
- **C2** (NEATINS) `ok` primește `1` / `true` / `0` / `false` amestecat (`:184`, `:190`, `:222`, `:246`), plus `rez = -1` ca steag la `:163` și `vf` int la `:277`. Recomandat: `bool` cu `true` / `false` peste tot.
- **C3** (nou) `teza1MI.h`: header gol (doar `#pragma once` + include), adăugat în `vcxproj` fără conținut. Ori îl umpli, ori nu-l comiți încă.
- **C4** (nou) `vectori.h:290`: `v[i]%10!=0` fără spații, spre deosebire de restul condiției.

Before / After (🔴)

Nicio 🔴 deschisă în această rundă. Cele patru închise, ca referință:

	Before (runda 1)	After (c56a122)
B1	`if (cif == 0) { break; }` + `ct == 3 && ogl != v[i]`	fără `break`; `ct == 3 && v[i] % 10 != 0 && ogl != v[i]`
B2	`ls = mij - 1;` și fără `else`	`ls = mij + 1;` + `else ld = mij - 1;`
B3	`i > 0`	`i >= 0`
B4	`int v[501]`	`int v[1000]`

Întrebări

1. La B2 reparat, pentru `x = 0`: ce valori iau `ls`, `ld` și `mij` la fiecare pas pe vectorul `{1,3,4,5,6,12,16,32}` și de ce se oprește bucla?
2. La problema 7, `d = 7` cu 8 valori scrise: de ce programul nu dă nicio eroare și pur și simplu nu afișează `87`? Ce conține `v[7]` și de ce nu-l vede nimeni?
3. La problema 3, dacă citești numerele unul câte unul din `numere.in` direct în `f`, de câtă memorie ai nevoie pentru 1 000 000 de numere de cel mult două cifre, comparat cu acum?

Runda 1 — 2026-10-16 — sha 5f145ef

- ✅ B1 închis — oglinda, un 0 la mijloc oprea numărarea (`vectori.h:282-284`)
- ✅ B2 închis — căutarea binară se bloca (`vectori.h:187-194`, `solutie8alternativ`)
- ✅ B3 închis — căutarea liniară sărea peste `v[0]` (`vectori.h:165`, `solutie8`)
- ✅ B4 închis — inserarea ieșea din vector (`vectori.h:117`, `solutie6`)
- 🟡 M1 `d = 7` la 8 valori (`vectori.h:140`) — NEATINS (vezi runda 2)
- 🟡 M2 niciun `fstream` — NEATINS
- 🟡 M3 `v[100]` la problemele 3, 4, 5 — NEATINS
- 🟡 M4 `main()` cheamă o singură funcție — REPETAT
- 🟡 M5 problema 2 cu `N = 0` — NEATINS
- 🟢 C1 `int x = d;` — NEATINS
- 🟢 C2 `ok` cu `1` / `true` / `0` / `false` amestecat — NEATINS